

Probabilidades de distribuciones Binomiales y Poisson

Matías González

Antonia Ruz

Profesor: Felipe Osorio – Métodos estadísticos

Objetivo: implementar algoritmos en Python que permitan el cálculo de probabilidades para las distribuciones discretas Binomiales y Poisson.

Algoritmos utilizados

El primer algoritmo implementado fue un generador de probabilidades binomiales con base en dos parámetros p y n mediante un *ciclo for* recursivo. Siempre y cuando p se ubique en el intervalo $(0,1)$ el algoritmo lo interpretará como la probabilidad de éxito del ensayo de Bernoulli, y siempre y cuando n sea un entero igual o mayor a 0, se lo utilizará como el número de ensayos. Como retorno, la función ***Binomial(p,n)***, entregará la lista de números cuyo índice k representa la probabilidad del k -ésimo ensayo.

Figura 1. Distribución Binomial de probabilidad p y n términos

$$\mathbb{P}(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}.$$

El segundo algoritmo implementado fue un generador de probabilidades de distribución Poisson a partir de un parámetro x y n mediante un *ciclo for* recursivo. Siendo x la media y varianza de la distribución siempre y cuando sea mayor a 0, y n el número de términos finitos a generar. Como retorno, la función ***Poisson(x,n)***, entregará una lista de números cuyo índice k representa la probabilidad de que ocurra un evento un número k de veces.

Figura 2. Distribución Poisson con media y varianza lamda

$$f(k; \lambda) = P(X=k) = \frac{\lambda^k e^{-\lambda}}{k!},$$

El tercer algoritmo implementado fue una distribución acumulada para variables discretas formuladas en una lista. A partir de un parámetro L , x y un opcional parámetro ***comp*** (fijado por defecto a “menor”). Siempre y cuando la cantidad de elementos de la lista L sea mayor o igual al parámetro x (cuyas entradas deben ser números reales mayores a 0) se utilizará L como la lista de probabilidades y sumará todas cuyo índice de lista sea igual o menor al parámetro x mediante un *ciclo for*. Si se modifica el parámetro ***comp*** a “mayor”, entonces el retorno corresponde al complemento ($P^c = 1 - P$), que corresponde a la suma de 1 y la suma calculada por el ciclo recursivo multiplicada por -1, representando la probabilidad de que la variable tenga una observación de valor estrictamente mayor al parámetro x . En otros casos, devuelve la suma calculada por el ciclo recursivo, que representa la probabilidad de que la observación sea igual o menor al parámetro x .

Figura 3. Función de distribución acumulada para variables aleatorias discretas

$$F(x) = P[X \leq x] = \sum_{u \leq x} f(u)$$

Resultados

A partir de los algoritmos anteriormente descritos, se calcularon las probabilidades indicadas por la siguiente tabla:

Tabla 1. Resultados a partir de algoritmos propios implementado en Python

Probabilidad	Distribución	Valor numérico
$P(140 \leq X \leq 200)$	$X \sim \text{Bin}(1000, 1/6)$	0.9881136443887650
$P(X \geq 350)$	$X \sim \text{Bin}(500, 1/4)$	-0.0000000000000009
$P(X \geq 50)$	$X \sim \text{Poi}(55)$	0.7677952194991420
$P(X \leq 350)$	$X \sim \text{Poi}(325)$	0.9200932740057400

Debido a la presencia de un valor negativo, se utilizaron las funciones *pbinom* y *ppois* del paquete *stats* de R para calcular valores y hacer comparativa del error numérico en los resultados. La probabilidad $P(X \geq 350)$, $X \sim \text{Bin}(500, 1/4)$ se considera un cero numérico debido a la aproximación utilizada en los cálculos.

Tabla 2. Resultados a partir de funciones del paquete *stats* de R

Probabilidad	Distribución	Valor numérico
$P(140 \leq X \leq 200)$	$X \sim \text{Bin}(1000, 1/6)$	0.9881136443887400
$P(X \geq 350)$	$X \sim \text{Bin}(500, 1/4)$	0.0000000000000000
$P(X \geq 50)$	$X \sim \text{Poi}(55)$	0.7677952194991430
$P(X \leq 350)$	$X \sim \text{Poi}(325)$	0.9200932740057240

Con esto se construyó una tabla comparativa para evaluar el error absoluto mediante Excel (definido por la figura 4).

Figura 4. Error absoluto

$$E_a = |P_R - P_{\text{Python}}|$$

Tabla 3. Comparativa y error numérico de los resultados en los distintos algoritmos

Probabilidad	Distribución	Valor en R (stats)	Valor en Python (propio)	Error absoluto
$P(140 \leq X \leq 200)$	$X \sim \text{Bin}(1000, 1/6)$	0.9881136443887400	0.9881136443887650	2.50E-14
$P(X \geq 350)$	$X \sim \text{Bin}(500, 1/4)$	0.0000000000000000	-0.0000000000000009	8.88E-16
$P(X \geq 50)$	$X \sim \text{Poi}(55)$	0.7677952194991430	0.7677952194991420	9.99E-16
$P(X \leq 350)$	$X \sim \text{Poi}(325)$	0.9200932740057240	0.9200932740057400	1.60E-14

De lo que se se encontró que, trabajando a 14 cifras, los 4 resultados del algoritmo implementado en Python coinciden con los valores de las funciones *pbinom* y *ppois* del paquete *stats*.

Referencias

Paquete stats utilizado: R Core Team. (2025). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.r-project.org/>